Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea practică Nr. 7

Disciplina: Tehnici de programare

Tema: ” Arsenalul sub lupă: GitHub și unealta reparată cu git ”

A efectuat : st. gr. SI – 263 Berejnoi Alexandr
A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Învățarea lucrului cu git și GitHub și repararea, cu git, a unei unelte Python stricate:

- **Lucrarea A** – git configurat pe computer și două repo. publice de exercițiu, `exercitiu-git` și `exercitiu-2`, cu mai multe commit-uri trimise pe GitHub (clone, add, commit, push, pull, `.gitignore`, log, diff);
- **Lucrarea B** – tool stricat, reparat cu git: scriptul `inventar.py` (rezumatul țintelor dintr-un fișier JSON) a fost adus, eroare cu eroare, la ieșirea așteptată, cu câte un commit după fiecare reparație; istoria din git a devenit jurnalul depanării.

Fișierele `inventar.py` și `tinte.json` sunt fișierele de laborator primite de la asistent. Unealta a fost reparată, nu rescrisă.

DESFĂȘURAREA LUCRĂRII

Fiecare etapă a lucrării este însoțită de captura comenzilor din terminal sau a paginii GitHub corespunzătoare. Istoria lucrării se află pe GitHub.

1. Pregătirea mediului

A1. Contul GitHub

Pe github a fost creat contul, cu adresa de e-mail citită zilnic, o parolă și un nume de utilizator profesional; adresa a fost confirmată cu codul primit pe e-mail. Numele de utilizator apare în toate linkurile depozitelor. GitHub recomandă și autentificarea în doi pași (2FA), cu o aplicație de autentificare pe telefon.

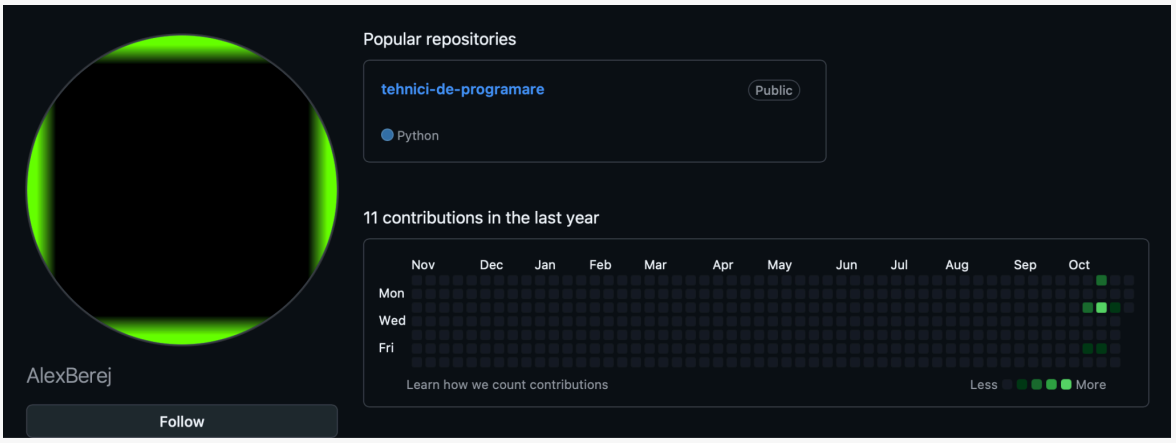


Figura 1.1 - Contul GitHub: profilul cu numele de utilizator

A2. Git pe PC

Versiunea git a fost verificată cu `git --version`. Apoi git a fost „prezentat”: `user.name` și `user.email` (adresa din contul GitHub, ca să lege commit-urile de cont). Setările au fost verificate cu `git config --global --list`.



Figura 1.2 - Versiunea git instalată



Figura 1.3 - Configurarea git (git config --global --list)

2. Lucrarea A. GitHub, pas cu pas

Git ține istoria proiectului pe calculator, iar GitHub ține o copie a ei. `origin` este numele scurt al copiei de pe GitHub. Toate comenzile git de mai jos au fost date în folderul depozitului.

A3. Primul repozitoriu, creat pe GitHub

Pe GitHub a fost creat `exercitiu-git` (New repository): descriere opțională, tip Public, cu opțiunea Add a README file bifată. Pagina depozitului arată fișierul `README.md`.

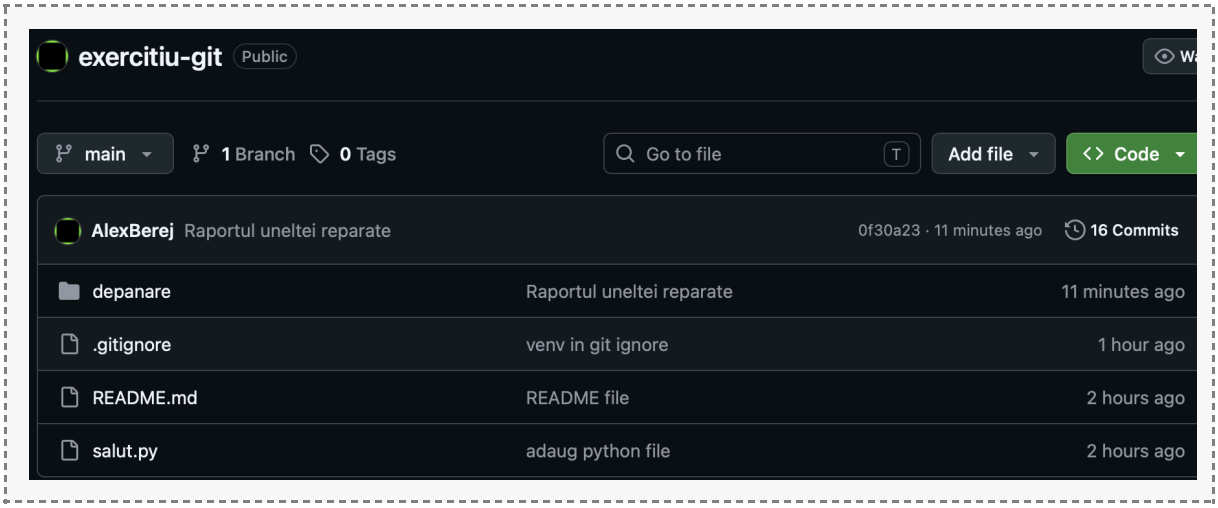


Figura 2.1 - Depozitul exercitiu-git creat pe GitHub

A4. Aducerea pe calculator: git clone

Adresa HTTPS din butonul verde Code a fost folosită în `git clone`. După clonare s-a intrat în folderul `exercitiu-git`, unde se dau toate comenzile git, iar `dir` (pe Linux `ls`) a arătat `README.md`.

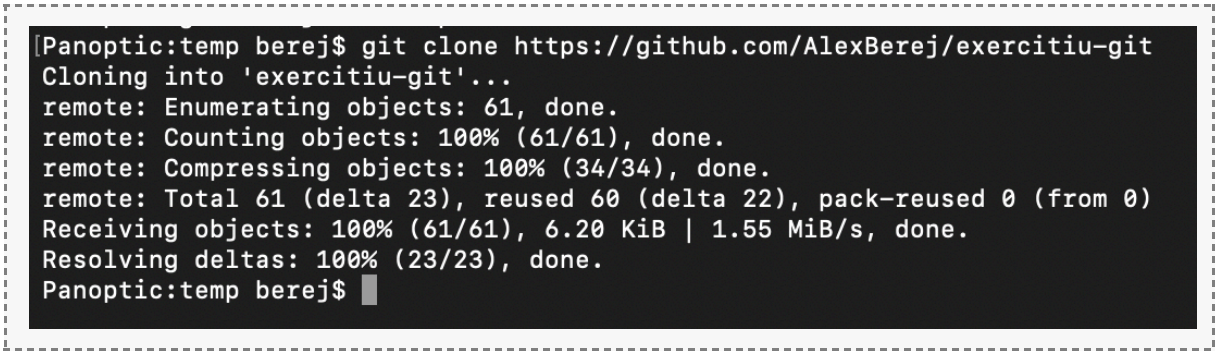


Figura 2.2 - Clonarea depozitului (git clone)

A5. Prima fotografie: status, add, commit

A fost creat fișierul `salut.py`, cu un singur rând, `print("Salut, GitHub!")`, și a fost rulat. `git status` îl arată ca „Untracked”, `git add salut.py` îl pune în fotografie, iar `git commit -m "Adaug salut.py"` face fotografia, cu un mesaj care spune ce s-a schimbat.

```
[Panoptic:tehnici-de-programare berej$ mkdir L7
[Panoptic:tehnici-de-programare berej$ cd L7
[Panoptic:L7 berej$ nano salut.py
[Panoptic:L7 berej$ python3 salut.py
 Salut, GitHub!
[Panoptic:L7 berej$ cat salut.py
 print("Salut, GitHub!")
 Panoptic:L7 berej$
```

Figura 2.3 - Fișierul salut.py rulat

```
[Panoptic:L7 berej$ git status salut.py
 On branch main
 Your branch is up to date with 'origin/main'.

 Untracked files:
   (use "git add <file>..." to include in what will be committed)
         salut.py

 nothing added to commit but untracked files present (use "git add" to track)
[Panoptic:L7 berej$ git status
 On branch main
 Your branch is up to date with 'origin/main'.

 Untracked files:
   (use "git add <file>..." to include in what will be committed)
         ../L6/arsenal/forensics/fisa.json
         ../L6/gasit.zip
         ../L6/probe/ascuns.png
         ../L6/probe/challenge.bin
         ../L6/probe/challenge_hint.txt
         ../L6/probe/foto.jpg
         ../L6/probe/hashuri.txt
         ../L6/probe/mesaj.txt
         ../L6/probe/wordlist.txt
         ../L6/probe/xor.bin
         ../L6/stego.png
         ./

 nothing added to commit but untracked files present (use "git add" to track)
 Panoptic:L7 berej$
```

Figura 2.4 - git status și git add

```
[Panoptic:L7 berej$ git add salut.py
[Panoptic:L7 berej$ git status salut.py
 On branch main
 Your branch is up to date with 'origin/main'.

 Changes to be committed:
   (use "git restore --staged <file>..." to unstage)
         new file:   salut.py

[Panoptic:L7 berej$ git commit -m "Adaug salut.py"
 [main 1543c2e] Adaug salut.py
  1 file changed, 1 insertion(+)
  create mode 100644 L7/salut.py
 Panoptic:L7 berej$
```

Figura 2.5 - Primul commit (git commit)

A6. Trimiterea pe GitHub: git push

Commit-ul a fost trimis cu `git push`. La prima rulare git cere autentificarea la GitHub. Pe macOS, în locul parolei contului, se folosește un token personal. După reîncărcarea paginii depozitului, `salut.py` apare cu mesajul „Adaug salut.py”.

```
[Panoptic:L7 berej$ git push origin main
 Username for 'https://github.com': AlexBerej
[Password for 'https://AlexBerej@github.com':
 fatal: failed to store: -60008
 Enumerating objects: 5, done.
 Counting objects: 100% (5/5), done.
 Delta compression using up to 8 threads
 Compressing objects: 100% (2/2), done.
 Writing objects: 100% (4/4), 346 bytes | 346.00 KiB/s, done.
 Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
 remote: Resolving deltas: 100% (1/1), completed with 1 local object.
 To https://github.com/AlexBerej/tehnici-de-programare
    10393c9..1543c2e  main -> main
 Panoptic:L7 berej$
```

Figura 2.6 - Trimiterea commit-ului pe GitHub (git push)



Figura 2.7 - Fișierul salut.py pe GitHub

A7. O schimbare făcută pe GitHub, adusă înapoi: git pull

În browser, `README.md` a fost editat: s-a adăugat un rând, apoi Commit changes. Calculatorul nu avea această fotografie, așa că a fost adusă cu `git pull` (Fast-forward)). Înainte de lucru `git pull`, după lucru `git push`.



Figura 2.8 - Editarea README.md pe GitHub

```
[Panoptic:L7 berej$ cd ..
[Panoptic:tehnici-de-programare berej$ git pull
 remote: Enumerating objects: 5, done.
 remote: Counting objects: 100% (5/5), done.
 remote: Compressing objects: 100% (3/3), done.
 remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
 Unpacking objects: 100% (3/3), 997 bytes | 199.00 KiB/s, done.
 From https://github.com/AlexBerej/tehnici-de-programare
    1543c2e..9679bdf  main       -> origin/main
 Updating 1543c2e..9679bdf
 Fast-forward
  README.md | 4 +++-
  1 file changed, 3 insertions(+), 1 deletion(-)
[Panoptic:tehnici-de-programare berej$ cat README.md
 # tehnici-de-programare

 Aici gasiti lucrarile la TP
 Panoptic:tehnici-de-programare berej$
```

Figura 2.9 - Aducerea schimbării pe calculator (git pull)

A8. Ce nu urcă pe GitHub: .gitignore

Mediul virtual a fost creat cu `python -m venv .venv`. Fișierul `.gitignore`, cu `.venv/` și `__pycache__/`, scoate mediul din listă.

```
[Panoptic:tehnici-de-programare berej$ python3 -m venv .venv
```

Figura 2.10 - Mediul virtual văzut de git

```
[Panoptic:tehnici-de-programare berej$ cat .gitignore
 .venv
 Notes.md
 *.DS_Store*
[*__pycache__*Panoptic:tehnici-de-programare berej$
[Panoptic:tehnici-de-programare berej$ git status .gitignore
 On branch main
 Your branch is up to date with 'origin/main'.

 nothing to commit, working tree clean
 Panoptic:tehnici-de-programare berej$
```

Figura 2.11 - Regula .gitignore salvată și trimisă

A9. Istoria și schimbările: log și diff

`git log --oneline` arată istoria proiectului, câte un rând pe commit. După adăugarea unui al doilea rând în `salut.py`, `git diff` a arătat schimbarea

înainte de commit (rândul cu + este adăugat, cel cu - ar fi șters). `git restore salut.py` a întors fișierul la ultima fotografie, iar `git status` a confirmat „working tree clean”.

```
[Panoptic:tehnici-de-programare berej$ git log --oneline
9679bdf (HEAD -> main, origin/main, origin/HEAD) Update README with project description
1543c2e Adaug salut.py
10393c9 Lab 6: decoder universal + forensics si stego
b45a025 Atestare 1
b800633 creare fisiere
d3e4ebc update gitignore
12b7541 pregatire atestare
6ad148b venv in gitignore
65663d7 test git
3623e2b codul la lucrarea 5
37e8418 initial commit
51bb5db Initial commit
Panoptic:tehnici-de-programare berej$
```

Figura 2.12 - Istoria depozitului (git log --oneline)

```
[Panoptic:L7 berej$ nano salut.py
[Panoptic:L7 berej$ git diff
diff --git a/L7/salut.py b/L7/salut.py
index e700475..396bbab 100644
--- a/L7/salut.py
+++ b/L7/salut.py
@@ -1 +1,2 @@
 print("Salut, GitHub!")
+# Linia 2
Panoptic:L7 berej$
```

Figura 2.13 - Schimbarea necommituită (git diff)

```
[Panoptic:L7 berej$ git restore salut.py
[Panoptic:L7 berej$ git status salut.py
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
Panoptic:L7 berej$
```

Figura 2.14 - Întoarcerea la ultima fotografie (git restore)

A10. Al doilea depozit, pornit de pe calculator

De data aceasta s-a pornit invers: întâi folderul local `exercitiu-2`, cu `git init`, un `README.md` și primul commit. Apoi, pe GitHub, a fost creat depozitul public `exercitiu-2`, FĂRĂ opțiunea Add a README file, iar legătura s-a făcut cu `git remote add origin`, `git branch -M main` și `git push -u origin main`. Opțiunea `-u` leagă ramura locală de cea de pe GitHub o singură dată; de acum încolo ajunge `git push`.

```
[Panoptic:git berej$ ls
 tehnici-de-programare
[Panoptic:git berej$ mkdir exercitiu-2
[Panoptic:git berej$ cd exercitiu-2/
[Panoptic:exercitiu-2 berej$ git init
 Initialized empty Git repository in /Users/berej/git/exercitiu-2/.git/
[Panoptic:exercitiu-2 berej$ nano README.md
[Panoptic:exercitiu-2 berej$ cat README.md
 # Al doilea depozit, si salvezi
[Panoptic:exercitiu-2 berej$ git add README.md
[Panoptic:exercitiu-2 berej$ git commit -m "Primul commit"
 [main (root-commit) 1b33f1b] Primul commit
  1 file changed, 1 insertion(+)
  create mode 100644 README.md
 Panoptic:exercitiu-2 berej$
```

Figura 2.15 - Depozitul local exercitiu-2 (git init)

```
[Panoptic:exercitiu-2 berej$ git remote add origin https://github.com/AlexBerej/exercitiu-2
[Panoptic:exercitiu-2 berej$ git branch -M main
[Panoptic:exercitiu-2 berej$ git push -u origin main
 Username for 'https://github.com': AlexBerej
[Password for 'https://AlexBerej@github.com':
 fatal: failed to store: -60008
 Enumerating objects: 3, done.
 Counting objects: 100% (3/3), done.
 Writing objects: 100% (3/3), 255 bytes | 255.00 KiB/s, done.
 Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
 To https://github.com/AlexBerej/exercitiu-2
  * [new branch]      main -> main
 branch 'main' set up to track 'origin/main'.
 Panoptic:exercitiu-2 berej$
```

Figura 2.16 - Legarea de GitHub (git push -u)

A11. Verificarea finală

`exercitiu-git` conține `README.md`, `salut.py` și `.gitignore`, iar în istorie sunt cel puțin 4 commit-uri; `exercitiu-2` conține `README.md`. Ambele depozite sunt publice. Linkuri: https://github.com/AlexBerej/exercitiu-git și https://github.com/AlexBerej/exercitiu-2

```
[Panoptic:git berej$ ls
 exercitiu-2             tehnici-de-programare
[Panoptic:git berej$ git clone https://github.com/AlexBerej/exercitiu-git
 Cloning into 'exercitiu-git'...
 warning: You appear to have cloned an empty repository.
[Panoptic:git berej$ cd exercitiu-git/
[Panoptic:exercitiu-git berej$ nano README.md
[Panoptic:exercitiu-git berej$ nano salut.py
[Panoptic:exercitiu-git berej$ touch .gitignore
[Panoptic:exercitiu-git berej$ git add README.md
[Panoptic:exercitiu-git berej$ git commit -m "README file:
[>
[Panoptic:exercitiu-git berej$ git commit -m "README file"
 [main (root-commit) 2b17d60] README file
  1 file changed, 1 insertion(+)
  create mode 100644 README.md
[Panoptic:exercitiu-git berej$ git add salut.py
[Panoptic:exercitiu-git berej$ git commit -m "adaug python file"
 [main 216d7d2] adaug python file
  1 file changed, 3 insertions(+)
  create mode 100644 salut.py
[Panoptic:exercitiu-git berej$ git add .gitignore
[Panoptic:exercitiu-git berej$ git commit -m ".gitignore empty"
 [main f78c69f] .gitignore empty
  1 file changed, 0 insertions(+), 0 deletions(-)
  create mode 100644 .gitignore
[Panoptic:exercitiu-git berej$ nano .gitignore
[Panoptic:exercitiu-git berej$ git add .gitignore
[Panoptic:exercitiu-git berej$ git commit -m "venv in git ignore"
 [main e963f5b] venv in git ignore
  1 file changed, 1 insertion(+)
[Panoptic:exercitiu-git berej$ git push origin main
 Username for 'https://github.com': AlexBerej
[Password for 'https://AlexBerej@github.com':
 fatal: failed to store: -60008
 Enumerating objects: 12, done.
 Counting objects: 100% (12/12), done.
 Delta compression using up to 8 threads
 Compressing objects: 100% (7/7), done.
 Writing objects: 100% (12/12), 1.06 KiB | 1.06 MiB/s, done.
 Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
 remote: Resolving deltas: 100% (1/1), done.
 To https://github.com/AlexBerej/exercitiu-git
  * [new branch]      main -> main
 Panoptic:exercitiu-git berej$
```

Figura 2.17 (a) - Depozitul exercitiu-git pe GitHub

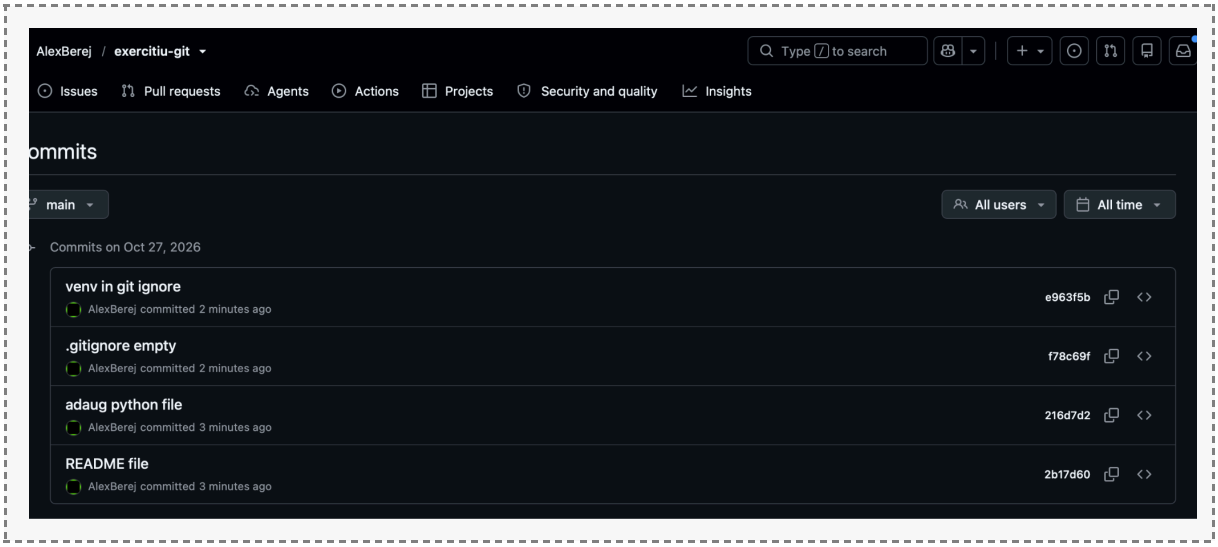


Figura 2.17 (b) - Depozitul exercitiu-git pe GitHub

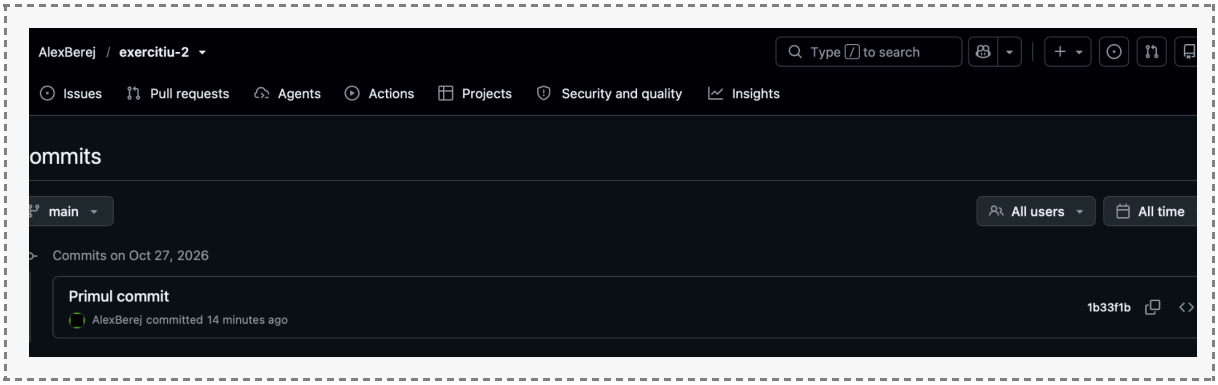


Figura 2.18 - Depozitul exercitiu-2 pe GitHub

3. Lucrarea B. Unealta stricată, reparată cu git

Unealta `inventar.py` primeste țintele dintr-un fișier JSON (`tinte.json`): numără țintele citite, grupează porturile pe servicii și listează gazdele cu un port peste un prag (`--prag`). A fost reparată câte o greșeală pe rând, cu un commit după fiecare reparație; ieșirea așteptată are 6 rânduri, de la „Tinte citite: 6” până la „Raport salvat in raport_inventar.json”.

B1. Pregătirea și starea de pornire

În `exercitiu-git` s-a rulat `git pull`, s-a creat folderul `depanare/` cu `inventar.py` și `tinte.json`, iar starea de pornire a fost salvată cu commit-ul „Pornire: inventar.py stricat”, ca să se vadă de unde s-a plecat. Prima rulare, `python inventar.py tinte.json --prag 1000`, se oprește imediat cu eroare.



Figura 3.1 - Starea de pornire și prima rulare a uneltei stricate

B2. Metoda de depanare

La fiecare eroare s-a procedat la fel: (1) se citește ultima linie a traceback-ului, adică tipul erorii și mesajul; (2) se merge la linia indicată; (3) se formulează o bănuială într-o propoziție; (4) se schimbă UN singur lucru și se rulează din nou; (5) când eroarea a dispărut, se face un commit care spune ce s-a reparat. Mesajul commit-ului este rândul din jurnal; s-a folosit `git commit -am`, care face add și commit într-un singur pas pentru fișierele deja urmărite.

B3. Greșelile care nu lasă programul să pornească

Repararea 1 – SyntaxError. Linia `def incarca(cale)` nu avea „:” la sfârșit, iar Python refuză să pornească fișierul (`SyntaxError: expected ':'`). A fost adăugat „:”. Commit: „SyntaxError: lipsea : la def incarca”.

```
[Panoptic:depanare berej$ nano inventar.py
[Panoptic:depanare berej$ git diff inventar.py
diff --git a/depanare/inventar.py b/depanare/inventar.py
index 4757f61..43301fb 100644
--- a/depanare/inventar.py
+++ b/depanare/inventar.py
@@ -5,7 +5,7 @@ import argparse
 import json


-def incarca(cale)
+def incarca(cale):
     """Citeste lista de tinte din fisierul JSON."""
     with open(cale) as f:
         return json.load(f)
[Panoptic:depanare berej$ git commit -am "SyntaxError: lipsea : la def incarca"
[main 9143e40] SyntaxError: lipsea : la def incarca
 1 file changed, 1 insertion(+), 1 deletion(-)
Panoptic:depanare berej$
```

Figura 3.2 - Repararea 1: SyntaxError

Repararea 2 – IndentationError. În `port_valid`, rândul `return None` de după `except ValueError:` nu era indentat (`IndentationError: expected an indented block after 'except'`). Rândul a fost indentat. Commit: „IndentationError: return None indentat sub except”.

```
[Panoptic:depanare berej$ python3 inventar.py tinte.json --prag 1000
  File "/Users/berej/git/exercitiu-git/depanare/inventar.py", line 19
    return None
    ^
IndentationError: expected an indented block
[Panoptic:depanare berej$ nano inventar.py
[Panoptic:depanare berej$ git diff inventar.py
diff --git a/depanare/inventar.py b/depanare/inventar.py
index 43301fb..0fc5533 100644
--- a/depanare/inventar.py
+++ b/depanare/inventar.py
@@ -16,7 +16,7 @@ def port_valid(text):
     try:
         return int(text)
     except ValueError:
-    return None
+        return None


 def pe_servicii(tinte):
[Panoptic:depanare berej$ git add inventar.py
[Panoptic:depanare berej$ git commit -am "IndentationError: return None indentat sub except"
[main 1df100e] IndentationError: return None indentat sub except
 1 file changed, 1 insertion(+), 1 deletion(-)
Panoptic:depanare berej$
```

Figura 3.3 - Repararea 2: IndentationError

B4. Greșelile de nume, de tip și de date

Repararea 3 – NameError. În `main`, `len(tinta)` folosea un nume care nu există; numele creat este `tinte` (`NameError: name 'tinta' is not defined`). Commit: „NameError tinta: numele corect e tinte”.

```
[Panoptic:depanare berej$ python3 inventar.py tinte.json --prag 1000
Traceback (most recent call last):
  File "/Users/berej/git/exercitiu-git/depanare/inventar.py", line 67, in <module>
    main()
  File "/Users/berej/git/exercitiu-git/depanare/inventar.py", line 52, in main
    print("Tinte citite: " + len(tinta))
NameError: name 'tinta' is not defined
[Panoptic:depanare berej$ nano inventar.py
[Panoptic:depanare berej$ git diff inventar.py
diff --git a/depanare/inventar.py b/depanare/inventar.py
index 0fc5533..d8fc76c 100644
--- a/depanare/inventar.py
+++ b/depanare/inventar.py
@@ -49,7 +49,7 @@ def main():
     a = p.parse_args()

     tinte = incarca(a.fisier)
-    print("Tinte citite: " + len(tinta))
+    print("Tinte citite: " + len(tinte))

     grupe = pe_servicii(tinte)
     for serviciu, porturi in grupe.items():
[Panoptic:depanare berej$ git add inventar.py
[Panoptic:depanare berej$ git commit -am "NameError tinta: numele corect e tinte"
[main cc107b9] NameError tinta: numele corect e tinte
 1 file changed, 1 insertion(+), 1 deletion(-)
Panoptic:depanare berej$
```

Figura 3.4 - Repararea 3: NameError

Repararea 4 – TypeError la concatenare. `"Tinte citite: " + len(tinte)` lipește text cu un număr (`TypeError: can only concatenate str (not "int") to str`). Rândul a fost scris cu f-string: `f"Tinte citite: {len(tinte)}"`. Commit: „TypeError la concatenare: f-string pentru len(tinte)”.

```
[Panoptic:depanare berej$ python3 inventar.py tinte.json --prag 1000
Traceback (most recent call last):
  File "/Users/berej/git/exercitiu-git/depanare/inventar.py", line 67, in <module>
    main()
  File "/Users/berej/git/exercitiu-git/depanare/inventar.py", line 52, in main
    print("Tinte citite: " + len(tinte))
TypeError: can only concatenate str (not "int") to str
[Panoptic:depanare berej$ nano inventar.py
[Panoptic:depanare berej$ git diff inventar.py
diff --git a/depanare/inventar.py b/depanare/inventar.py
index d8fc76c..888c686 100644
--- a/depanare/inventar.py
+++ b/depanare/inventar.py
@@ -49,7 +49,7 @@ def main():
     a = p.parse_args()

     tinte = incarca(a.fisier)
-    print("Tinte citite: " + len(tinte))
+    print(f"Tinte citite: " + len(tinte))

     grupe = pe_servicii(tinte)
     for serviciu, porturi in grupe.items():
[Panoptic:depanare berej$ git add inventar.py
[Panoptic:depanare berej$ git commit -am "TypeError la concatenare: f-string pentru len(tinte)"
[main e0d37af] TypeError la concatenare: f-string pentru len(tinte)
 1 file changed, 1 insertion(+), 1 deletion(-)
Panoptic:depanare berej$
```

Figura 3.5 - Repararea 4: TypeError la concatenare

Repararea 5 – KeyError. Ultima țintă din `tinte.json` nu are cheia `serviciu` (`KeyError: 'serviciu'`). Accesul `t["serviciu"]` a devenit `t.get("serviciu", "necunoscut")`. Commit: „KeyError serviciu: t.get cu valoarea necunoscut”.

```
[Panoptic:depanare berej$ python3 inventar.py tinte.json --prag 1000
Traceback (most recent call last):
  File "/Users/berej/git/exercitiu-git/depanare/inventar.py", line 67, in <module>
    main()
  File "/Users/berej/git/exercitiu-git/depanare/inventar.py", line 52, in main
    print(f"Tinte citite: " + len(tinte))
TypeError: can only concatenate str (not "int") to str
[Panoptic:depanare berej$ nano inventar.py
[Panoptic:depanare berej$ git diff
diff --git a/depanare/inventar.py b/depanare/inventar.py
index 888c686..066cf86 100644
--- a/depanare/inventar.py
+++ b/depanare/inventar.py
@@ -23,7 +23,7 @@ def pe_servicii(tinte):
     """Dict: serviciu -> lista de porturi."""
     grupe = {}
     for t in tinte:
-        serviciu = t["serviciu"]
+        serviciu = t.get("serviciu", "necunoscut")
         port = port_valid(t["port"])
         if port is None:
             print("sar peste", t["gazda"], ": port invalid", t["port"])
[Panoptic:depanare berej$ git add inventar.py
[Panoptic:depanare berej$ git commit -am "KeyError serviciu: t.get cu valoarea necunoscut"
[main 93aa0fa] KeyError serviciu: t.get cu valoarea necunoscut
 1 file changed, 1 insertion(+), 1 deletion(-)
Panoptic:depanare berej$
```

Figura 3.6 - Repararea 5: KeyError

Repararea 6 – AttributeError (NoneType). Funcția `pe_servicii` nu avea `return`, așa că `main` primea `None` (`AttributeError: 'NoneType' object has no attribute 'items'`). La final a fost adăugat `return grupe`. Commit: „AttributeError NoneType: pe_servicii intoarce grupe”.

```
[Panoptic:depanare berej$ python3 inventar.py tinte.json --prag 1000
Tinte citite: 6
sar peste 192.168.56.103 : port invalid abc
Traceback (most recent call last):
  File "/Users/berej/git/exercitiu-git/depanare/inventar.py", line 67, in <module>
    main()
  File "/Users/berej/git/exercitiu-git/depanare/inventar.py", line 55, in main
    for serviciu, porturi in grupe.items():
AttributeError: 'NoneType' object has no attribute 'items'
[Panoptic:depanare berej$ nano inventar.py
[Panoptic:depanare berej$ git diff inventar.py
diff --git a/depanare/inventar.py b/depanare/inventar.py
index 066cf86..66e832e 100644
--- a/depanare/inventar.py
+++ b/depanare/inventar.py
@@ -30,7 +30,7 @@ def pe_servicii(tinte):
             continue
         grupe[serviciu] = []
         grupe[serviciu].append(port)
-
+    return grupe

 def peste_prag(tinte, prag):
     """Lista gazdelor cu un port mai mare decat pragul."""
@@ -49,7 +49,7 @@ def main():
     a = p.parse_args()

     tinte = incarca(a.fisier)
-    print(f"Tinte citite: " + len(tinte))
+    print(f"Tinte citite: {len(tinte)}")

     grupe = pe_servicii(tinte)
     for serviciu, porturi in grupe.items():
[Panoptic:depanare berej$ git add inventar.py
[Panoptic:depanare berej$ git commit -am "AttributeError NoneType: pe_servicii intoarce grupe"
[main 813c688] AttributeError NoneType: pe_servicii intoarce grupe
 1 file changed, 2 insertions(+), 2 deletions(-)
Panoptic:depanare berej$
```

Figura 3.7 - Repararea 6: AttributeError (NoneType)

Repararea 7 – TypeError la comparație. Valoarea `--prag 1000` ajunge din terminal ca text, iar `port > prag` compară un număr cu un text (`TypeError: '>' not supported between instances of 'int' and 'str'`). Argumentul a primit `type=int`. Commit: „TypeError la prag: type=int in argparse”.

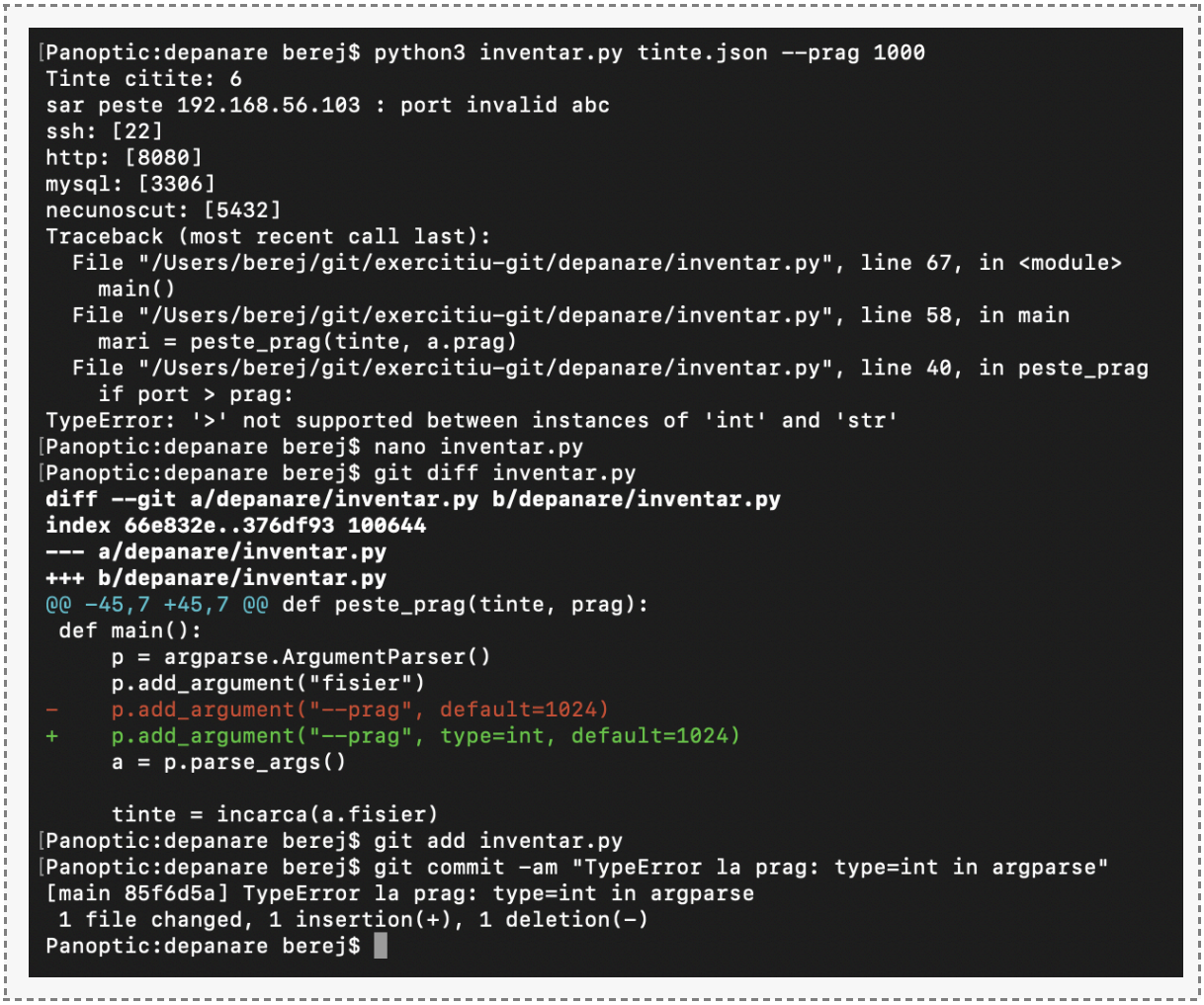


Figura 3.8 - Repararea 7: TypeError la comparație

B5. Greșelile tăcute

După repararea 7 unealta nu mai crapă, dar ieșirea nu este cea așteptată. Comparată rând cu rând, ea arată două greșeli fără eroare și, după prima dintre ele, apare încă o eroare de rulare.

Repararea 8 – rezultat gol. Ieșirea conținea `Peste 1000: []`. Rândul `return rezultat` din `peste_prag` era indentat în bucla `for`, deci funcția se oprea după prima țintă. Rândul a fost mutat în afara buclei. Commit: „Rezultat gol: return peste_prag scos din bucla for”.

```
[Panoptic:depanare berej$ python3 inventar.py tinte.json --prag 1000
 Tinte citite: 6
 sar peste 192.168.56.103 : port invalid abc
 ssh: [22]
 http: [8080]
 mysql: [3306]
 necunoscut: [5432]
 Peste 1000: []
 Raport salvat in raport_inventar.json
[Panoptic:depanare berej$ nano inventar.py
[Panoptic:depanare berej$ git diff inventar.py
diff --git a/depanare/inventar.py b/depanare/inventar.py
index 376df93..b311b97 100644
--- a/depanare/inventar.py
+++ b/depanare/inventar.py
@@ -39,7 +39,7 @@ def peste_prag(tinte, prag):
         port = int(t["port"])
         if port > prag:
             rezultat.append(t["gazda"])
-        return rezultat
+    return rezultat


 def main():
[Panoptic:depanare berej$ git add inventar.py
[Panoptic:depanare berej$ git commit -am "Rezultat gol: return peste_prag scos din bucla for"
 [main 00fbaa1] Rezultat gol: return peste_prag scos din bucla for
  1 file changed, 1 insertion(+), 1 deletion(-)
 Panoptic:depanare berej$
```

Figura 3.9 - Repararea 8: rezultat gol (Peste 1000: [])

Repararea 9 – ValueError. Acum bucla parcurge toate țintele și ajunge la portul „abc” (`ValueError: invalid literal for int() with base 10: 'abc'`), fiindcă `peste_prag` apela `int()` direct. A fost folosită funcția `port_valid`, deja scrisă, cu verificarea `port is not None`. Commit: „ValueError abc: peste_prag foloseste port_valid”.

```
[Panoptic:depanare berej$ python3 inventar.py tinte.json --prag 1000
Tinte citite: 6
sar peste 192.168.56.103 : port invalid abc
ssh: [22]
http: [8080]
mysql: [3306]
necunoscut: [5432]
Traceback (most recent call last):
  File "/Users/berej/git/exercitiu-git/depanare/inventar.py", line 67, in <module>
    main()
  File "/Users/berej/git/exercitiu-git/depanare/inventar.py", line 58, in main
    mari = peste_prag(tinte, a.prag)
  File "/Users/berej/git/exercitiu-git/depanare/inventar.py", line 39, in peste_prag
    port = int(t["port"])
ValueError: invalid literal for int() with base 10: 'abc'
[Panoptic:depanare berej$ nano inventar.py
[Panoptic:depanare berej$ git diff inventar.py
diff --git a/depanare/inventar.py b/depanare/inventar.py
index b311b97..f977df2 100644
--- a/depanare/inventar.py
+++ b/depanare/inventar.py
@@ -36,8 +36,8 @@ def peste_prag(tinte, prag):
     """Lista gazdelor cu un port mai mare decat pragul."""
     rezultat = []
     for t in tinte:
-        port = int(t["port"])
-        if port > prag:
+        port = port_valid(t["port"])
+        if port is not None and port > prag:
             rezultat.append(t["gazda"])
     return rezultat

[Panoptic:depanare berej$ git add inventar.py
[Panoptic:depanare berej$ git commit -am "ValueError abc: peste_prag foloseste port_valid"
[main d0e425c] ValueError abc: peste_prag foloseste port_valid
 1 file changed, 2 insertions(+), 2 deletions(-)
Panoptic:depanare berej$
```

Figura 3.10 - Repararea 9: ValueError

Repararea 10 – lista căreia îi lipsește un element. Ieșirea arăta `http: [8080]` în loc de `http: [80, 8080]`: `grupe[serviciu] = []` golea lista la fiecare țintă. Cele două rânduri au fost înlocuite cu `grupe.setdefault(serviciu, []).append(port)`. Commit: „Lista http pierdea 80: setdefault in loc de grupe[serviciu] = []”.

```
[Panoptic:depanare berej$ python3 inventar.py tinte.json --prag 1000
 Tinte citite: 6
 sar peste 192.168.56.103 : port invalid abc
 ssh: [22]
 http: [8080]
 mysql: [3306]
 necunoscut: [5432]
 Peste 1000: ['192.168.56.102', '192.168.56.103', '192.168.56.104']
 Raport salvat in raport_inventar.json
[Panoptic:depanare berej$ nano inventar.py
[Panoptic:depanare berej$ git diff inventar.py
diff --git a/depanare/inventar.py b/depanare/inventar.py
index f977df2..82fa5ce 100644
--- a/depanare/inventar.py
+++ b/depanare/inventar.py
@@ -28,8 +28,7 @@ def pe_servicii(tinte):
          if port is None:
              print("sar peste", t["gazda"], ": port invalid", t["port"])
              continue
-        grupe[serviciu] = []
-        grupe[serviciu].append(port)
+        grupe.setdefault(serviciu, []).append(port)
      return grupe

  def peste_prag(tinte, prag):
[Panoptic:depanare berej$ git add inventar.py
[Panoptic:depanare berej$ git commit -am "Lista http pierdea 80: setdefault in loc de grupe[serviciu] = []"
[main 47416e0] Lista http pierdea 80: setdefault in loc de grupe[serviciu] = []
 1 file changed, 1 insertion(+), 2 deletions(-)
Panoptic:depanare berej$
```

Figura 3.11 - Repararea 10: lista porturilor completă

B6. Verificarea finală și push

Unealta a fost rulată cu `--prag 1000` și cu pragul implicit (1024); ieșirea coincide cu cea așteptată. Raportul `raport_inventar.json` a fost adăugat într-un commit separat, „Raportul uneltei reparate”, și trimis cu `git push`. `git log --oneline` arată, de jos în sus, pornirea, cele zece reparații și raportul; același istoric se vede pe GitHub.

```
[Panoptic:depanare berej$ python3 inventar.py tinte.json --prag 1000
 Tinte citite: 6
 sar peste 192.168.56.103 : port invalid abc
 ssh: [22]
 http: [80, 8080]
 mysql: [3306]
 necunoscut: [5432]
 Peste 1000: ['192.168.56.102', '192.168.56.103', '192.168.56.104']
 Raport salvat in raport_inventar.json
[Panoptic:depanare berej$ python3 inventar.py tinte.json
 Tinte citite: 6
 sar peste 192.168.56.103 : port invalid abc
 ssh: [22]
 http: [80, 8080]
 mysql: [3306]
 necunoscut: [5432]
 Peste 1024: ['192.168.56.102', '192.168.56.103', '192.168.56.104']
 Raport salvat in raport_inventar.json
 Panoptic:depanare berej$
```

Figura 3.12 - Rularea uneltei reparate

```
inventar.py           raport_inventar.json  tinte.json
Panoptic:depanare berej$ cat raport_inventar.json
{
  "servicii": {
    "ssh": [
      22
    ],
    "http": [
      80,
      8080
    ],
    "mysql": [
      3306
    ],
    "necunoscut": [
      5432
    ]
  },
  "peste_prag": [
    "192.168.56.102",
    "192.168.56.103",
    "192.168.56.104"
  ]
}Panoptic:depanare berej$
```

Figura 3.13 - Conținutul fișierului raport_inventar.json

```
Panoptic:depanare berej$ git add raport_inventar.json
Panoptic:depanare berej$ git commit -m "Raportul uneltei reparate"
[main 0f30a23] Raportul uneltei reparate
 1 file changed, 22 insertions(+)
 create mode 100644 depanare/raport_inventar.json
Panoptic:depanare berej$ git push origin main
Username for 'https://github.com': AlexBerej
Password for 'https://AlexBerej@github.com':
fatal: failed to store: -60008
Enumerating objects: 50, done.
Counting objects: 100% (50/50), done.
Delta compression using up to 8 threads
Compressing objects: 100% (49/49), done.
Writing objects: 100% (49/49), 5.26 KiB | 2.63 MiB/s, done.
Total 49 (delta 21), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (21/21), done.
To https://github.com/AlexBerej/exercitiu-git
   e963f5b..0f30a23  main -> main
Panoptic:depanare berej$
```

Figura 3.14 - Raportul salvat și trimis pe GitHub

```
Panoptic:depanare berej$ git log --oneline -13
0f30a23 (HEAD -> main, origin/main, origin/HEAD) Raportul uneltei reparate
47416e0 Lista http pierdea 80: setdefault in loc de grupe[serviciu] = []
d0e425c ValueError abc: peste_prag foloseste port_valid
00fbaa1 Rezultat gol: return peste_prag scos din bucla for
85f6d5a TypeError la prag: type=int in argparse
813c688 AttributeError NoneType: pe_servicii intoarce grupe
93aa0fa KeyError serviciu: t.get cu valoarea necunoscut
e0d37af TypeError la concatenare: f-string pentru len(tinte)
cc107b9 NameError tinta: numele corect e tinte
1df100e IndentationError: return None indentat sub except
9143e40 SyntaxError: lipsea : la def incarca
56c22f0 Pornire: inventar.py stricat
e963f5b venv in git ignore
Panoptic:depanare berej$
```

Figura 3.15 - Jurnalul depanării (git log --oneline)

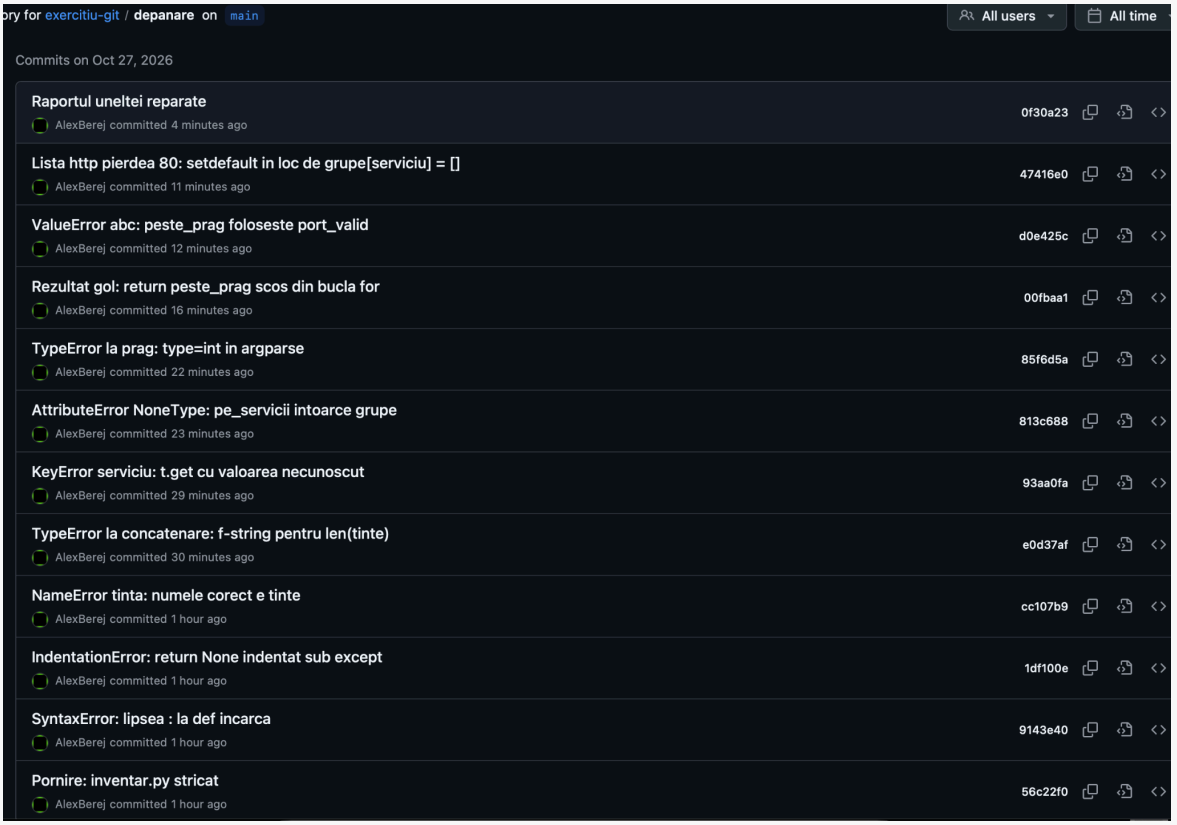


Figura 3.16 - Istoria și folderul depanare/ pe GitHub

Rezultate

Sarcina	Rezultat
Cont GitHub și git	cont: AlexBerej; git configurat: `user.name`, `user.email`, `init.defaultbranch=main`, `pull.rebase=false`
exercitiu-git	Public; `README.md`, `salut.py`, `.gitignore`; cel puțin 4 commit-uri (A3–A9)
exercitiu-2	Public; pornit cu `git init` și trimis cu `git push -u origin main` (A10)
Comenzi folosite	`clone`, `status`, `add`, `commit`, `push`, `pull`, `log`, `diff`, `restore`
Ce nu urcă pe GitHub	`.venv/` și `__pycache__/` (fișierul `.gitignore`)
inventar.py reparat	10 greșeli, 10 commit-uri: 2 de formă (SyntaxError, IndentationError), 6 de rulare (NameError, TypeError ×2, KeyError, AttributeError, ValueError), 2 tăcute (rezultat gol, listă golită)
Ieșirea uneltei	`ssh: [22]`; `http: [80, 8080]`; `mysql: [3306]`; `necunoscut: [5432]`; peste 1000: `192.168.56.102`, `192.168.56.103`, `192.168.56.104`
Istoria din depanare/	pornirea, cele zece reparații și raportul (`raport_inventar.json`)
Linkuri	https://github.com/AlexBerej/exercitiu-git și https://github.com/AlexBerej/exercitiu-2

CONCLUZII

În această lucrare am învățat să lucrez cu git și GitHub. Am creat un cont, am configurat git pe calculator și am făcut două repozitorii publice. Am folosit ciclul `status` – `add` – `commit` – `push`, am adus schimbările de pe GitHub cu `git pull`, am exclus fișierele care nu trebuie urcate cu `.gitignore` și am citit istoria și schimbările cu `git log` și `git diff`. În a doua parte am reparat unealta `inventar.py`: zece greșeli, câte un commit pentru fiecare.

GitHub ține istoria pe calculator, iar GitHub ține o copie a ei online; de aceea se face `git pull` înainte de lucru și `git push` după. Cele mai greu de găsit au fost greșelile tăcute, care nu dau eroare și se descoperă doar comparând ieșirea cu cea așteptată (UB). Un commit cu mesaj clar după fiecare reparație face din istoria git un jurnal al depanării. Din punctul de vedere al securității, tokenul de acces se păstrează ca o parolă, iar într-un depozit public nu se pun parole sau chei.

BIBLIOGRAFIE

1. https://metanit.com/python/tutorial/5.1.php
2. https://git-scm.com/docs/git-restore
3. https://docs.python.org/3/tutorial/errors.html
4. Fișa de laborator „Laboratorul 8. Arsenalul sub lupă, Lucrarea A. GitHub, pas cu pas și Lucrarea B. Unealta stricată, reparată cu git”, Tehnici de programare, UTM, FCIM.